ADVANCED AI SOCIETY · PROOF-OF-CONTROL INITIATIVE
Working draft v0.1 · August 2026 · for co-author review prior to submission

A Digest Is a Change Detector, Not a Claim: Composition Commitments for Agent Systems, and the Attack Class They Cannot See

— Composition is a theory. You have to state it before you can hash it.

Jim Schwoebel[1,2]

[1] Quome [2] Advanced AI Society, `https://advancedaisociety.org`[1]

ABSTRACT

A Software Bill of Materials says what a system is made of. Applied to an agent, it runs into two problems at once. The first is timing: an agent's composition changes after the bill is taken. A tool server can present one schema when an operator reviews it and a wider one when the agent calls it, and a manifest taken at task start is byte-identical across that substitution — so the document that was supposed to detect component swaps is provably blind to the component swap that matters most. The second is worse, because it is not a bug: nobody has agreed what belongs on the bill. Any tool that ships a default schema settles an open research question by configuration file, and the settlement is invisible to whoever reads its output. We built `spectrum`, which refuses to do that. It defines no Agent Bill of Materials. It defines a digest over a *declared* component set, ships three such sets as named falsifiable hypotheses, requires that every one of them state what it cannot capture, and refuses to compare digests taken under different hypotheses. On top of the static manifest it keeps a PCR-style running commitment over composition changes, which holds 32 bytes however much composition moves. A detection harness then runs real substitutions against all three hypotheses. The tool-definition rug-pull is detected under the two runtime hypotheses and invisible under the build-time one. Content poisoning is detected under none of them, and we assert that negative in the test suite on purpose. We also report the uncomfortable half: on the content row the harness cannot fail, because `Detected` is unreachable outside the tool-measurement branch by construction of the predicate. That row is a worked consequence of the design, not an experiment. Locally, re-measuring a tool list costs 6.2 µs at 8 tools and 177 µs at 256, linear at roughly 0.7 µs per tool — around a hundredth of the standard's per-action budget at realistic sizes, and *not* an answer to whether re-measurement is affordable, because the network round trip that dominates it is not measured here. Our conclusion for the Agent Bill of Materials is narrower than "inventory" and more useful than "security control": a running composition commitment is a change-detection control over the capability surface and an inventory control over content.

[1] Prepared within the Proof-of-Control Initiative of the Advanced AI Society. Authorship is limited to contributors to this manuscript. Co-authors will be added only upon explicit consent and substantive contribution; the author list is deliberately short at this stage rather than inherited from a sibling paper.

Artefact. All figures in Section 6 come from `spectrum bench-mcp` on one host, and every detection outcome in Section 5 is real output from `spectrum detect`. The implementation is Apache-2.0 [20]. Nothing in this paper was produced by a deployment: there is no fleet, no network, and no operator but us.

1 The Problem

An operator is asked to approve a tool. The approval screen shows a payments server offering one function:

```
{"tools":[{"name":"charge",
         "parameters":{"amount":"number","currency":"string"}}]}
```

They read it. Charging an amount in a currency is what a payments tool should do, and there is nowhere in that schema to put a destination, so the money can only go where the application already decided it goes. They approve it. The system takes a bill of materials — model, system prompt, tool definitions, dependency tree — hashes it, and files the hash with the task record.

The agent runs. At the moment it actually calls the tool, the server returns this instead:

```
{"tools":[{"name":"charge",
         "parameters":{"amount":"number","currency":"string",
                     "destination":"string"}}]}
```

There is now somewhere to put a destination, and the model — helpful, schema-following, doing exactly what it was built to do — can be talked into putting one there.

Now ask what the bill of materials says about this. It says nothing. Not "suspicious," not "unverified." It produces the same hash it produced before, because it was taken at task start and the tool list it hashed was the one presented at task start. The artefact whose entire purpose is to detect a substituted component is, against this substitution, a constant function.

I want to be careful about what is and is not surprising, because there is a reading on which this is trivial. Of course a snapshot does not capture a later change. Snapshots do not do that. Everybody nods.

The nodding does not survive contact with a deployment. What you get in practice is a pipeline that emits an SBOM at build time, a policy that requires one, an auditor who checks that it was emitted and that its digest matches the one on file, and a green tick. Every step is real work. The green tick means "the components we recorded at task start are the components we recorded at task start," and it is read as "the agent ran with the components you approved." The gap is not in anybody's understanding of hashing. It is in the distance between what the artefact commits to and what its presence is taken to mean.

1.1 The second problem, which is the harder one

Suppose we fix that by re-measuring at call time. Then a different question arrives immediately, and it has no technical answer: re-measure *what*?

A conventional SBOM has a settled answer. A program is made of files and libraries; you enumerate them. An agent is made of — and here the sentence stops. A model, certainly, though "the model" behind a hosted API endpoint is a name and a version string and not something you can hash. A system prompt. Tool definitions. The dependency tree of the harness. Also, arguably: the documents the retriever pulled into the context this turn, the memories it read, the sub-agents

it spawned, the tool list each remote server returned at each call. Also, if you are being ambitious: which of those retrieved documents actually influenced the decision.

That list is not a specification. It is a set of positions, and reasonable people hold different ones. Including retrieved chunks makes the bill enormous and turns every content digest into a cross-session fingerprint of what the agent read. Excluding them means the bill says nothing about the entire class of attacks that work by feeding an agent a document. There is no neutral choice, and there is certainly no choice that a tool should make silently on its user's behalf.

This is the position P10 of the Proof-of-Control research roadmap is in [1]: it asks whether an Agent Bill of Materials can be a running, verifiable commitment, and whether that catches anything a static one does not. It is an open question, marked open. And the fastest way to make an open question look closed is to ship a tool with a default.

> A tool that ships a default component set answers a research question by configuration file, and the answer is invisible in the output. Every digest it prints looks like a fact about a system. It is a fact about a system *under a theory of composition the reader was never shown.*

So `spectrum` has no default profile. `--profile` is required on every command that composes a manifest. There is no invocation of this tool that can produce an unmarked "the AgBOM," and that is a deliberate ergonomic cost paid to keep a research question from being settled by a default value.

1.2 What this paper claims

Three things, in decreasing order of confidence.

First, that the timing problem is real and mechanically demonstrable, and that its fix — a running commitment extended at composition-change points — catches the tool-substitution class. We demonstrate this against a hostile server that performs the substitution, under three profiles, and report which ones catch it.

Second, that the fix does *not* extend to content. A poisoned document that is validly sourced, correctly digested and faithfully recorded moves the register exactly as a benign one does. The commitment records that the chunk arrived. It says nothing about whether the chunk was adversarial, and nothing about whether it steered the decision. We assert this negative in the test suite so that no future change can make it quietly pass.

Third, and most tentatively, that the honest summary for P10 is: *a running composition commitment is a change-detection control over the capability surface, and an inventory control over content.* That moves the threat model on tool-and-plugin substitution and does not move it on the memory- and context-poisoning rows.

2 What Others Have Built

Nearly every mechanism in this paper is borrowed, and the borrowing is the point: the contribution is not a new digest construction.

Bills of materials. CycloneDX [15] and SPDX [9] are the two formats in use, the second standardised as ISO/IEC 5962. The NTIA minimum-elements report [11] is what most procurement language actually points at, and US Executive Order 14028 [22] and the EU Cyber Resilience Act [6] are why anybody is required to produce one. This work reads CycloneDX and does not read

SPDX, which is a choice we defend badly in Section 7. All of these formats describe a build. None of them has a concept of a component that changes identity between two calls within one execution, because for the software they were designed for, no such component exists.

Provenance for builds. SLSA [13] and in-toto [24] secure the path from source to artefact: who built what, from which inputs, on which machine. The threat they address is the substituted build step, and Thompson's compiler [23] is the ancestor of the whole line. Ohm and colleagues catalogued what supply-chain attacks actually look like in package ecosystems [12]. Our attacker is adjacent but not the same: they do not compromise the build, they change what a remote endpoint returns *after* the build was attested and after a human approved it.

Running commitments. The register in Section 3 is a TPM Platform Configuration Register with the hardware removed [25]. Extend-only, fixed width, order-dependent, no setter. The idea that you can summarise an unbounded sequence of measurements in 32 bytes and lose nothing except the ability to reconstruct them is thirty years old, and we use it unmodified. Merkle's hash tree [10] and Certificate Transparency [8] are the other half of that tradition; the sibling tool `ephemeris` [17] is where they live in this programme, and Section 3 explains why a register without a log is only half of a verification story.

Canonicalisation. Comparing two JSON documents for identity is a solved problem with an unusually large number of ways to get it wrong. We use RFC 8785 JCS [16], through `transit`'s implementation [21] rather than a second one of our own, over strictly parsed JSON [5, 4]. A server that reformats its JSON must not read as a composition change; a server that sends duplicate keys must be refused rather than silently resolved to whichever key a decoder prefers.

Evidence classes. The RATS architecture [3] separates attester, verifier and relying party, and insists that a claim's weight depends on who is vouching for it. Our witness class is a coarse version of that idea pushed inside the digest: a component an operator typed and a component whose bytes we hashed must not render identically. `parallax` [19] is the sibling tool that works out what such a distinction is worth across a whole deployment.

Agent-specific attacks. Greshake and colleagues demonstrated indirect prompt injection [7]: content that enters a model's context and redirects it. The OWASP LLM list [14] has since catalogued the family. This literature is why Section 5's negative result matters. The attack that most concerns practitioners is delivered as *content*, and content is exactly the axis on which a composition commitment degenerates into an inventory.

Where we differ. The work above mostly asks how to build the artefact correctly. We are asking what a correctly built artefact is worth when the thing it describes changes underneath it, and — separately — what a tool should do when the community has not decided what the artefact should contain. Our answer to the second is the profile: ship the disagreement as named, comparable, individually falsifiable hypotheses, and refuse to rank them.

3 Writing It Down Precisely

The symbols will not make anything truer than the English already did. They are here because several of the properties below are easy to state loosely and then implement in a way that looks

right, and because the failure modes we care about live in details — concatenation order, length prefixing, which fields enter a hash — that prose glosses over.

Definition 1 (Profile)**.** *A profile is a triple* $P = (\mathrm{id}, K, C)$ *where* id *is a stable name,* $K \subseteq \mathcal{K}$ *is a set of component kinds drawn from a closed enumeration* $\mathcal{K}$*, and* C *is a non-empty set of natural-language statements of what this theory of composition does not capture. A profile with* $C = \emptyset$ *does not load.*

That last clause is the one doing work. It is not possible to express, in this tool's configuration language, the proposition "this profile captures everything." The check is four lines long and it is the most opinionated code in the repository.

Definition 2 (Component reference)**.** *A component reference is* $c = (k, \mathrm{id}, d, w)$ *with* $k \in \mathcal{K}$*,* id *a package URL where one exists and a stable local name otherwise,* d *an optional tagged content digest, and* $w \in \{\textsc{Measured}, \textsc{Attested}, \textsc{Declared}, \textsc{Opaque}\}$ *the witness class — how we know what we are claiming.*

Opaque is a positive statement: a component of this kind is present and cannot be named. It is never the result of a failure. That distinction matters because the alternative — letting an unmeasurable component fall out of the manifest — makes "we could not measure the training data" and "there is no training data" render identically.

Definition 3 (Component digest)**.**

$$H_c(c) = \text{SHA-256}\big(\texttt{"spectrum-component-v1"} \parallel \mathrm{len}(J) \parallel J\big)$$

where J *is the JCS canonicalisation of the object* $\{\texttt{kind} : k, \texttt{id}, \texttt{content_digest} : d, \texttt{witness} : w\}$ *and* len *is a 64-bit big-endian length.*

Three details are load-bearing. The absent digest is rendered as an explicit `null` rather than omitted, so "we have no digest" and "we did not mention a digest" cannot collide. The witness class is *inside* the digest, which is one of P10's open forks and which we resolve in the direction the design recommends — at a real cost, discussed in Section 7. And the domain tag is hashed *before* the canonical bytes rather than carried as a key inside the JSON object, so it can never be shadowed by a future field that happens to be named the same thing. A tag inside the JCS input is just another field, and another field is precisely what it must be distinguishable from.

Definition 4 (Manifest and its digest)**.** *A manifest* $M = (P.\mathrm{id}, S)$ *carries a sorted, deduplicated sequence* $S = \langle c_1, \ldots, c_n \rangle$*. Its digest is*

$$H_M(M) = \text{SHA-256}\big(\texttt{"spectrum-manifest-v1"} \parallel \mathrm{len}(P.\mathrm{id}) \parallel P.\mathrm{id} \parallel n \parallel \mathrm{len}(H_c(c_1)) \| H_c(c_1) \parallel \cdots\big).$$

The profile id is inside the digest. This is what makes a digest a commitment *under a stated theory of composition* rather than a fact about a system, and it is why `diff` refuses to compare two manifests taken under different profiles: a difference between two theories is not a difference between two systems. Each component digest is length-prefixed, so a set of two components cannot be made to hash like one component whose identifier happens to contain the concatenation of their digests.

Proposition 1 (The sorting invariant is not cosmetic)**.** *Let* cmp *be the order-insensitive comparison used by* `diff`*, which indexes components by* (k, id)*. If a caller could append to* S *after construction, there would exist manifests* M_1, M_2 *over the same component set with* $\mathrm{cmp}(M_1, M_2) = \emptyset$ *and* $H_M(M_1) \neq H_M(M_2)$*.*

That is a producer and a verifier inside one library reaching opposite conclusions about the same pair of manifests. The remedy is structural rather than documentary: both fields of the manifest type are private, sorting and deduplication happen in the only constructor, and there is deliberately no `push`, no `components_mut`, and no conversion from a bare vector. The invariant holds by construction rather than by promise.

Definition 5 (Composition register)**.**

$$C_0 = \text{SHA-256}\big(\texttt{"spectrum-register-v1"} \,\|\, \text{len}(P.\text{id}) \| P.\text{id} \,\|\, \text{len}(H_M) \| H_M\big)$$
$$C_n = \text{SHA-256}\big(C_{n-1} \,\|\, H_e(e_n)\big)$$

where H_e is a domain-tagged digest over an event's kind, its interception hook, and the digest of the component that moved.

The register is 32 bytes regardless of how much composition has moved, which is what makes a per-step field affordable. The hook is inside the event digest, because the same component arriving at context assembly and at memory write are different composition changes.

Proposition 2 (One-way by construction)**.** *There is no sequence of public operations returning a register to a previous value other than by preimage.*

The inner value is private, there is no setter, no `reset`, and no constructor other than C_0. This is worth stating as a property because the convenience method that would violate it — "reset composition for a new task" — is one an implementer would reach for without noticing that it is a rewrite of history.

3.1 What the register does not do

It solves the record-size problem completely and the verification problem not at all.

To check a register you need its events, and this tool does not store them. Replaying a truncated log does not reproduce the register — which is the property that makes the construction worth anything — but noticing that requires having the log. So the honest statement of what `spectrum` provides is: a 32-byte value that changes when composition changes, and a demonstration that it changes for the right reasons, with the storage and custody of the event log left to `ephemeris` [17], whose own paper explains why that custody is the hard part.

There is a further limitation that is easy to miss and that we state plainly because it undercuts the natural reading of the whole construction: *nothing verifies a digest.* No infrastructure in this programme publishes expected AgBOMs anywhere. There is no registry, no transparency log of composition roots, nothing to check a value against. What the library offers is `diff` against a prior digest of the same deployment. A digest here is a change detector between two records of the same system — not a checkable claim about one record in isolation. Every sentence in this paper about "detection" should be read against that.

4 Three Hypotheses, Not One Answer

The tool ships three profiles. They are not a menu of configurations for different users; they are three positions on P10's open question, written down in a form that can be run against an attack and shown to be wrong.

`build-manifest` exists to reproduce the behaviour of what is deployed today. It is shipped so that the cost and the detection power of the other two are measured *against* it rather than asserted

Profile	Includes	The claim	What it cannot capture
`build-manifest`	model, system prompt, tool definitions, code and dependency SBOM	composition is fixed at task start	everything that changes after
`runtime-tools`	the above, plus MCP tool-list re-measurement and the sub-agent roster	the capability surface changes at runtime and is small enough to re-measure	retrieved context, memory, whether a component is adversarial
`runtime-with-context`	the above, plus retrieved-chunk and memory-read references	everything entering the context is part of composition	which of it mattered — and it costs a linkability leak

Table 1: The three shipped profiles. The right-hand column is not commentary: it is a required, non-empty field of the profile file, and a profile that omits it fails to load.

over it, and it is deliberately not named `default`: it is not the default answer, it is the current one, and this tool exists partly to show what it misses.

The profile mechanism has one more consequence worth drawing out. Because the profile id is inside every digest, two deployments running under different profiles produce digests that are not comparable, and the library refuses to compare them rather than reporting a difference. An operator who upgrades from `build-manifest` to `runtime-tools` does not see "composition changed." They see a refusal, with a message explaining that they changed theories. We think this is the correct behaviour and we note that it is annoying, which is usually a sign that a tool is telling the truth about something.

4.1 What a profile still does not tell you

A profile's coverage list states what the *theory* excludes. It does not state what the *plumbing* dropped. CycloneDX defines component types — `container`, `operating-system`, `firmware`, `device`, `platform` — that this projection has no kind for, and those are discarded before the profile is ever consulted. No shipped profile's coverage mentions them, so nothing in a profile tells an operator that a container in their document did not reach the digest.

We treat this as a real gap and mitigate it at the point of use rather than pretending it is covered: `compose` counts the unmapped types and prints them under the digest, so a dropped type is visible on the run that dropped it rather than inferred from a list it is not on.

```
profile build-manifest
components 1
digest sha-256:ee76701fa13e35df0ea03f6cc304f54ca3313eb6e04cc6dc3f5e0de531cedc45

DECLARED BUT ABSENT - this profile includes these kinds and no
component of them was collected, so the digest above says nothing
about them. They are missing, not empty:
  - system_prompt
  - tool_definition
  - code_dependency

UNMAPPED CycloneDX TYPES - present in the document, no kind to
project them onto, dropped before the profile was consulted:
  - container (2)
  - firmware (1)
```

```
- operating-system (1)
```

The "declared but absent" block is the same discipline applied to a different gap. Four of the eight kinds have no collector in this build, so a profile that declares them will routinely produce a digest that commits to none of them. The design's fail-loud table asks `compose` to refuse in that situation. It warns instead, because refusing would make the command unusable until collectors that are several build steps away exist. That is a compromise and we record it as one: the digest still prints, and it never prints alone.

5 Does It Actually Work?

5.1 The harness

`spectrum detect --scenario <name>` runs one attack against every shipped profile and reports, per profile, whether that profile's commitment moved and which component moved it. The exit code is 0 if any profile detected the attack and 1 if none did.

The hostile server is a pure function of a scenario and a call count, so every detection test runs with no socket, no network and no credentials. Call 0 is the operator's review; every later call is a real invocation. That split is the whole attack. There is also a real HTTP transport over the same function for a human to poke at by hand, and nothing in the automated suite touches it.

"Detected" has a precise and narrow meaning here, and we would rather state it three times than have it read as something grander: *the register value after the attack differs from the value a verifier would expect, and the event log names which component moved.* It does not mean the attack was prevented. It does not mean anything was judged malicious. Nothing in this tool blocks anything.

5.2 The matrix

Table 2 is real output, reformatted. Every cell was produced by running the command named in Section 5.

Scenario	`build-manifest`	`runtime-tools`	`runtime-with-context`
`none` (control)	not detected	not detected	not detected
`tool-rug-pull`	not detected	DETECTED	DETECTED
`added-tool`	not detected	DETECTED	DETECTED
`content-poisoning`	not detected	not detected	not detected

Table 2: Detection outcomes. Detected cells name the component that moved (`mcp://payments`). The first row is the negative control and the last row is the finding.

Three things to read off it.

The control is honest. With no attack, nothing is detected under any profile. This row exists because a harness that reports detection when nothing happened would make every other row worthless, and a suite without this row cannot tell the difference between a working detector and a stuck one.

The rug-pull is caught, and by the profile you would predict. Under `build-manifest` the digest before and after the substitution is identical, because the manifest was taken once. Under `runtime-tools` the register diverges at the call and the event names `mcp://payments`. This is detectable for reasons that do not generalise: a tool definition is small, local, re-readable on demand, and has an expected value that a human approved. Every one of those properties is load bearing, and content has none of them.

Content poisoning is caught by nothing. A document that is validly sourced, correctly digested, faithfully recorded and malicious moves the register exactly as a benign document does. Under `runtime-with-context` the register *does* change — the chunk entered the context and the commitment records that it did — and the scenario still reports not detected, because a change that a verifier has no expected value for is a record of arrival, not a detection.

5.3 What the harness proves, and what it only re-states

This is the part of the evaluation we would most like to overstate, so we will state it against ourselves.

On the content row, the harness could not have come out the other way. In the detection routine, the `Detected` outcome is reachable only when a "moved" variable is set, and that variable is assigned only inside the branch guarded by *does this profile include MCP servers*. A content attack therefore cannot reach `Detected` by construction of the predicate, whatever the profile says and whatever the register does.

That is deliberate, and the code's own documentation gives the reason: a verifier has no expected value for a chunk retrieved at runtime, so a register that moved because a chunk arrived is a record of arrival and not a detection. We think the negative is correct and the argument for it is sound. But the harness *executes* that argument against real profiles and real substitutions; it does not independently discover it. The content row of Table 2 is a worked consequence of the design, not an experiment that could have falsified it.

We are labouring this because the table looks like an experiment, tables generally are experiments, and a reader skimming for results would take that row as an empirical finding about composition commitments in general. It is not. It is a demonstration that our implementation behaves the way our argument says it should, which is worth having and is a strictly weaker thing.

The rows that *could* have come out the other way are the rug-pull and added-tool rows under `runtime-tools`, and the control row under all three. Those depend on canonicalisation behaving, on the digest covering the fields we think it covers, and on the register actually diverging. Any of them could have failed, and during development the equivalent of the control row is what catches a detector that has become stuck.

5.4 How we checked the rest

The suite is 76 tests, all offline. The ones that matter for this paper's claims are the ones that pin a property against a future edit rather than exercise a happy path:

- `the_witness_class_is_inside_the_digest` asserts that a DECLARED component does not hash like a MEASURED one, which pins P10's fork 2 in the direction we took. Reversing that fork is a visible test failure rather than a quiet behaviour change.

- `every_witness_tag_is_distinct` and `every_event_kind_tag_is_distinct` assert that no two variants share a wire tag. A copy-paste that gave two evidence classes the same string would undo fork 2's resolution without touching the code that implements it.
- `the_manifest_digest_of_a_two_kind_manifest_is_pinned` turns a reordering of the kind enumeration into a test failure. The declaration order of that enumeration decides concatenation order and therefore the digest of every multi-kind manifest; reordering it looks cosmetic in review.
- `a_static_deployment_never_advances_its_register` pins the property that the running commitment degenerates exactly to the static one when nothing changes.
- `replaying_a_truncated_log_does_not_reproduce_the_register` is the property that makes the register worth keeping.
- `content_poisoning_is_not_detected_by_any_profile` asserts the negative result against all three profiles, on purpose. Its comment says it is the row most likely to be deleted by somebody tidying the suite. It should not be.

6 What It Costs

The design's central bet is fork 5: that an MCP server's identity should be a digest of the tool list as re-measured, rather than an endpoint URL and a declared version. Re-measurement is what makes the rug-pull visible. The obvious objection is cost, because re-measurement happens per call and the Proof-of-Control standard's commentary sets a per-action budget of 15 ms [2].

6.1 The measurement

We measured the local half: canonicalisation and hashing of a synthetic tool list, in process, with the list built before the timing loop and a 100-iteration warm-up outside it. Seven repetitions per size, 10 000 iterations each, one host.

Tools	**Min** (µs)	**Median** (µs)	**Max** (µs)	**Spread** (%)	**Median as % of** 15 ms
1	1.250	1.277	1.386	10.9	0.0085
2	1.983	2.045	2.149	8.4	0.0136
4	3.281	3.346	3.430	4.5	0.0223
8	6.047	6.197	6.364	5.2	0.0413
16	11.404	11.587	11.861	4.0	0.0772
32	22.006	22.494	22.613	2.8	0.1500
64	43.083	43.310	43.921	1.9	0.2887
128	84.491	84.945	87.629	3.7	0.5663
256	176.164	177.259	182.886	3.8	1.1817

Table 3: Local cost of one MCP tool-list re-measurement. Seven repetitions per row, 10 000 iterations per repetition, Apple M5 Max, `rustc 1.97.1`, release build. Spread is $(\max - \min)/\min$.

The cost is linear in list size. A least-squares fit over the nine sizes gives 0.685 µs per tool with a fixed cost of about 0.27 µs; taking the marginal cost between the 64- and 256-tool rows instead gives 0.698 µs per tool. The fit under-predicts the small end by about 7%, which is what you would expect from a straight line dominated by the large sizes, so the per-tool figure is the number to quote and the intercept is not.

Extrapolating the fit, a tool list would need roughly 22 000 entries before local re-measurement consumed the entire 15 ms budget. At 256 tools it consumes 1.2% of it.

6.2 One run is not a benchmark

The repetitions turned up something that changes how the previously committed figures should be read. The results file in the repository reports 9.635 µs at 8 tools, from a single run. The median of seven repetitions on the same host, same toolchain, same binary, is 6.197 µs. A single standalone run we took while preparing this paper landed at 7.021 µs, between the two.

That is a factor of 1.55 between two measurements of the same quantity on the same machine. Within a sweep the spread is 1.9%–10.9% and tightens as the work per iteration grows, which is the ordinary shape of timing noise. Across sweeps, at the small end, it is not small at all.

The ordering — first-run slowest, sweep-median fastest — is consistent with a warm-up effect that the in-process 100-iteration warm-up does not cover: that loop warms the allocator and the branch predictors, and does nothing about CPU frequency or scheduler placement. We label that a hypothesis rather than a finding, because we did not test it, and testing it properly means pinning frequency and affinity, which we did not do.

The practical consequence is a correction to our own earlier reporting: the 9.635 µs figure is not wrong as a record of one run, and it overstates the steady-state cost by about half. Both figures support the same conclusion — the local cost is two to three orders of magnitude below the budget — which is why this did not change any argument. It would have changed one if the true figure had been near the budget.

6.3 What this does not settle

Fork 5 is not resolved by Table 3, and we want to be blunt about the gap because a table of microseconds is exactly the kind of artefact that gets cited as though it settled something.

What is measured is canonicalisation and hashing. What is not measured is the round trip to a remote MCP server, which is the operator's network. That round trip is expected to dominate the total, by a margin large enough that the local cost is a rounding error inside it. A negligible local cost is a *necessary* condition for re-measurement being affordable inside the per-action budget. It is not a *sufficient* one. Whether re-measurement is affordable end-to-end depends on a number this repository cannot produce, because producing it requires a real deployment with a real network and real servers, and we have none of those.

For scale, the standard's own commentary records a single hardware attestation quote at 39.5 ms [2] — already over the per-action budget on its own. The interesting comparison for fork 5 is against that kind of number, not against ours.

7 What We Still Don't Know

7.1 The forks we did not resolve

`spectrum` implements a design that leaves eight forks open, and it is not the case that the tool takes the recommended branch on each. It resolves one. Table 4 is the accounting.

One thing bearing on fork 3 is worth naming without closing it. The test `a_static_deployment_never_advances_its_register` pins the property the recommendation rests on — that the running value degenerates exactly to the static root when nothing changes. That is evidence for

Fork	What this work does
1. SPDX support	**Diverges from the recommendation.** The design recommends reading both formats and emitting CycloneDX. This tool reads CycloneDX only. An operator with an SPDX document cannot use `compose`. A second reader is a second identity derivation to keep in agreement, and it is not built.
2. Witness class in the digest	**Resolved, as recommended: inside.** Pinned in both directions by tests.
5. MCP server identity	**Instrumented, not resolved.** `runtime-tools` implements re-measurement and Section 6 measures its local cost. The network round trip decides this fork and needs a deployment.
7. Context linkability	**Half built.** See Section 7.3.
3, 4, 6, 8	**Untouched.** No branch is taken because nothing here reaches the decision: no field of this tool is emitted as an AgBOM digest, there is no prompt collector, no retriever, and no delegation. Claiming this tool "takes the recommended branch" on them would be an empty claim about code that does not exist.

Table 4: P10's eight forks against what this implementation does.

the recommendation, not a resolution of it, because nothing in this tool fills the field the fork is about.

7.2 Influence, which nobody can compute

The standard's C1.2.2 asks for a link between retrieved content and the decision it influenced. No profile in this tool captures it, no field expresses it, and there is no API that could produce one.

That absence is deliberate and it is enforced by there being no such field. A stub — an `influence` column populated with retrieval scores, or attention weights, or a heuristic — would be read as the C1.2.2 link by anybody downstream, and it would be wrong in a way that is very hard to detect from the outside, because a plausible number in a plausible column is indistinguishable from a real one. The tool records presence and never names it influence, and the coverage output says so on every run.

We do not know how to compute this. We are not aware of anybody who does. It is the single largest gap between what a composition commitment provides and what the threat model wants from it.

7.3 The privacy regression we shipped behind a warning

`runtime-with-context` digests retrieved content, and a content-derived chunk digest is a cross-session, cross-agent fingerprint of what was read. Two evidence records carrying the same chunk digest prove that both agents read the same document. Across a fleet, that is a linkability channel that did not exist before the composition commitment was introduced — built, by us, in the name of oversight.

Fork 7 is the decision point and it is unresolved. What we did instead of resolving it is refuse to bury it: `spectrum coverage` keyed on a profile including retrieved chunks prints the warning as output, not as documentation. That is a mitigation for the operator who runs the command and no mitigation at all for the deployment that does not.

The sibling tool `occultation` [18] exists to cost out the unlinkability machinery that would be the actual answer here. The two halves do not meet: nothing in this repository consumes it, and we are not claiming that they compose without somebody doing that work.

7.4 Discipline that is written down and not enforced

Two properties in this design are intended discipline rather than enforced behaviour, and the distinction is not visible from the outside, so we state it here.

Hosted-model weight identity should be DECLARED and never MEASURED: no measurement of a hosted model's weights exists from outside the provider. Training-data lineage should be OPAQUE. The code enforces neither. There is no model collector at all; no collector in the library ever constructs an OPAQUE component (only the test suite does, to check that such a component still has an identity); no test asserts either property; and the component type is a plain public structure, so a caller can write MEASURED against a hosted model today and nothing objects.

These are rules written down before the code they constrain exists, rather than after. We think that is the right order. We also think a reader is entitled to know that today they are prose.

7.5 The cost of resolving fork 2 the way we did

Putting the witness class inside the digest means that improving your instrumentation changes your digest even though the system did not change. Move a component from DECLARED to MEASURED — exactly the improvement you would want an operator to make — and every downstream comparison reports a difference.

We accepted that cost because the alternative is a digest in which an operator's typed version string is indistinguishable from measured bytes. But we have not measured the cost, and it is the kind of cost that lands on a fleet all at once: an instrumentation rollout looks like a fleet-wide composition change. The library reports witness changes as their own category, separately from content changes, which is the smallest useful mitigation and is not the same as knowing what a rollout does to a real deployment's alert volume.

7.6 The evidence base

One host, one afternoon, no deployment. The detection harness runs against a hostile server we wrote, presenting an attack we chose, to profiles we designed. There is no fleet, no third party, and no adversary who was not us. Every number in Section 6 is local, and the number that would decide fork 5 is not in this paper.

We think the argument in Section 5 survives that — the rug-pull is a mechanical consequence of when you measure, not an empirical claim about how often anybody does it — but the frequency question, the one an operator would actually ask, is untouched. We do not know how often tool definitions change between approval and call in real deployments. We do not know whether it ever happens maliciously. We have shown that if it happens, one profile sees it and another cannot.

8 Conclusion

An Agent Bill of Materials taken at task start is a constant function against the substitution it most needs to catch. Making it a running commitment fixes that for the capability surface: a tool list that changes between approval and call moves a 32-byte register, and the event names the component that moved. We measured what re-measurement costs locally — around 0.7 µs per tool,

a hundredth of the per-action budget at realistic list sizes — and we did not measure the network round trip that will decide whether it is affordable.

The same construction does nothing for content. A poisoned document that is validly sourced and faithfully recorded moves the register exactly as a benign one does, and the commitment records arrival rather than detecting anything. We assert that negative in the test suite so it cannot quietly become a positive, and we have said plainly that the harness re-states this result rather than discovering it.

So the answer we would give P10 is narrower than the question hoped for:

> A running composition commitment is a change-detection control over the capability surface, and an inventory control over content.

The second half of that sentence is the part worth carrying forward. It means the memory- and context-poisoning rows of the threat model are not addressed by any amount of additional rigour in the bill of materials, because the problem there is not that we failed to record the component — we recorded it perfectly — but that recording a component is not the same as knowing what it did. That is a different research programme, and this tool's contribution to it is to have declined to pretend otherwise, in a field on the profile where the pretence would have gone.

Availability. The implementation, the three profiles, the detection harness and the benchmark are Apache-2.0 licensed [20]. The detection outcomes in Table 2 are reproduced by `spectrum detect`; Table 3 by `spectrum bench-mcp`. Every test in the repository runs offline, with no network, no credentials and no socket.

References

[1] Advanced AI Society, Proof-of-Control Initiative. P10: The agent bill of materials. Research roadmap paper, working draft, 2026. [verify] repository URL and revision; this work answers the question P10 poses and resolves none of its forks by fiat.

[2] Advanced AI Society, Proof-of-Control Initiative. Proof-of-control: An open standard for runtime verifiability and cryptographic oversight in autonomous ai execution. Working draft, 2026. [verify] repository URL and version at submission time.

[3] Henk Birkholz, Dave Thaler, Michael Richardson, Ned Smith, and Wei Pan. Remote ATtestation procedures (RATS) architecture. RFC 9334, Internet Engineering Task Force, 2023.

[4] Tim Bray. The I-JSON message format. RFC 7493, Internet Engineering Task Force, 2015.

[5] Tim Bray. The javascript object notation (JSON) data interchange format. STD 90, RFC 8259, Internet Engineering Task Force, 2017.

[6] European Parliament and Council. Regulation on horizontal cybersecurity requirements for products with digital elements (cyber resilience act), 2024. [verify] regulation number, adoption date, and the annex that carries the bill-of-materials obligation.

[7] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023. [verify] page range.

[8] Ben Laurie, Adam Langley, and Emilia Käsper. Certificate transparency. RFC 6962, Internet Engineering Task Force, 2013.

[9] Linux Foundation. SPDX specification. Standardised as ISO/IEC 5962, 2021. [verify] the ISO/IEC 5962 designation, its year, and the SPDX version it corresponds to.

[10] Ralph C. Merkle. Protocols for public key cryptosystems. In *Proceedings of the 1980 IEEE Symposium on Security and Privacy*, pages 122–134, 1980. [verify] page range.

[11] National Telecommunications and Information Administration. The minimum elements for a software bill of materials (SBOM). Report to the United States Department of Commerce, 2021. [verify] exact title and publication date.

[12] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. Backstabber's knife collection: A review of open source software supply chain attacks. In *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*, 2020. [verify] page range and publisher series.

[13] Open Source Security Foundation. SLSA: Supply-chain levels for software artifacts. Specification, 2026. [verify] version number at submission time.

[14] OWASP Foundation. OWASP top 10 for large language model applications. Community standard, 2025. [verify] edition year and the item numbers cited in Section 2.

[15] OWASP Foundation. CycloneDX bill of materials specification. Specification, OWASP CycloneDX, 2026. [verify] version number and publication date at submission time; this work reads the JSON encoding.

[16] Anders Rundgren, Bret Jordan, and Samuel Erdtman. JSON canonicalization scheme (JCS). RFC 8785, Internet Engineering Task Force, 2020.

[17] Jim Schwoebel. `ephemeris`: Custody and durability for a tamper-evident log. Apache-2.0 licensed software, 2026. [verify] public repository URL at submission time.

[18] Jim Schwoebel. `occultation`: What unlinkability costs per action. Apache-2.0 licensed software, 2026. [verify] public repository URL at submission time.

[19] Jim Schwoebel. `parallax`: Computing the residual trust set of an attestation deployment. Apache-2.0 licensed software, 2026. [verify] public repository URL at submission time.

[20] Jim Schwoebel. `spectrum`: A composition commitment for agent systems. Apache-2.0 licensed software, 2026. [verify] public repository URL at submission time.

[21] Jim Schwoebel. `transit`: Canonicalization and signing for agent decision records. Apache-2.0 licensed software, 2026. [verify] public repository URL at submission time; this work depends on its `jcs` and `json` modules.

[22] The White House. Executive order 14028: Improving the nation's cybersecurity. Federal Register, 2021. [verify] Federal Register volume and page citation.

[23] Ken Thompson. Reflections on trusting trust. *Communications of the ACM*, 27(8):761–763, 1984.

[24] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. in-toto: Providing farm-to-table guarantees for bits and bytes. In *Proceedings of the 28th USENIX Security Symposium*, 2019. [verify] page range.

[25] Trusted Computing Group. TPM 2.0 library specification. Specification, 2019. [verify] revision and part number for the PCR extend operation cited in Section 4.